



Day-1

Python Course

12/20/2025



Python for Beginners | Prepared by @shivscript

Agenda

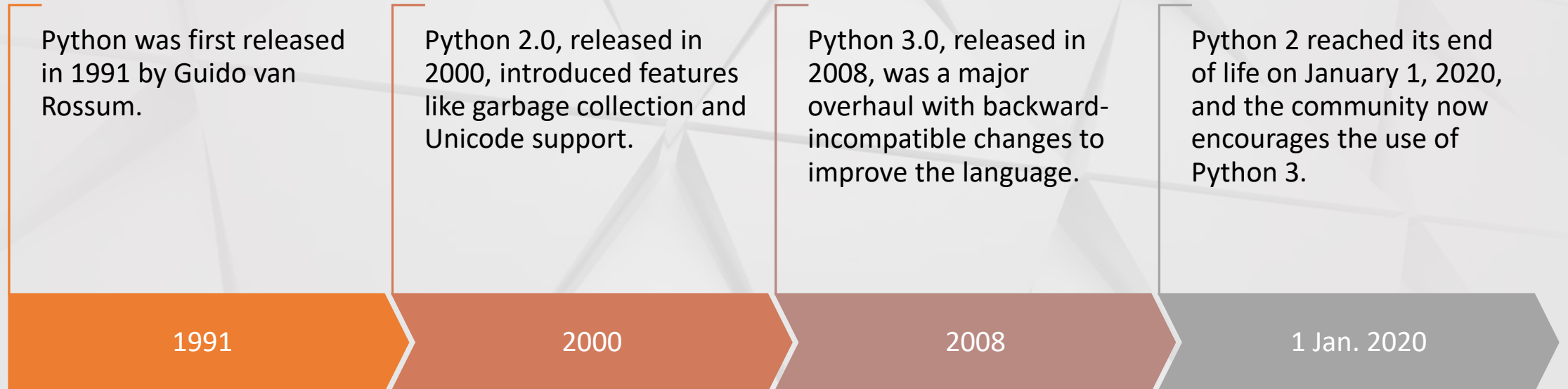
- Introduction
- History
- Features
- Applications
- Installation based on device
- IDE (Integrated Development Environment)

Introduction

- Python is a high-level, versatile, and interpreted programming language known for its readability and ease of use.
- Guido van Rossum initially developed it in the late 1980s, and it has since become one of the most popular programming languages in the world.
- Python is widely used for web development, data science, artificial intelligence, and automation.



History





Features

- **Readability:** Python's syntax is clear and readable, making it easy for beginners and experienced programmers alike.
- **Versatility:** Python supports both procedural and object-oriented programming paradigms, allowing developers to choose the approach that best suits their needs.
- **Extensive Libraries:** Python has a vast standard library and a rich ecosystem of third-party libraries, making it easy to access a wide range of functionalities.
- **Interpreted Language:** Python is an interpreted language, meaning that code can be executed line by line, which facilitates debugging and development.
- **Dynamically Typed:** Python is dynamically typed, meaning you don't need to specify variable types explicitly; the interpreter infers them at runtime.

Application

- **Web Development:** Frameworks like Django and Flask are popular for building web applications.
- **Data Science:** Python is widely used for data analysis, machine learning, and scientific computing with libraries like NumPy, Pandas, and scikit-learn.
- **Automation:** Python is used for automating repetitive tasks and scripting.
- **Artificial Intelligence:** Python has gained popularity in AI and machine learning with libraries like TensorFlow and PyTorch.
- **Desktop Applications:** GUI libraries like Tkinter and PyQt enable the development of desktop applications.

Installation based on device

- For Windows: Download the Python installer from the official website (<https://www.python.org/>) and run the installer, following the installation wizard.
- For macOS: Python is pre-installed on most Mac systems. Alternatively, you can use a package manager like Homebrew or download from the official website.
- For Linux: Python is often pre-installed. Use the package manager (e.g., apt or yum) to install Python. You can also compile from source.



IDE (Integrated Development Environment)

- **PyCharm:** A popular IDE developed by JetBrains with features like code completion, debugging, and project navigation.
- **VSCode:** A lightweight, open-source code editor with Python support and a vast extension ecosystem.
- **Jupyter Notebooks:** Ideal for data science and interactive computing, allowing users to create and share documents containing live code, equations, visualizations, and narrative text.
- **IDLE:** The default IDE that comes bundled with Python, providing a simple environment for writing and running Python code.

Running Hello World Program in Python





Day-2

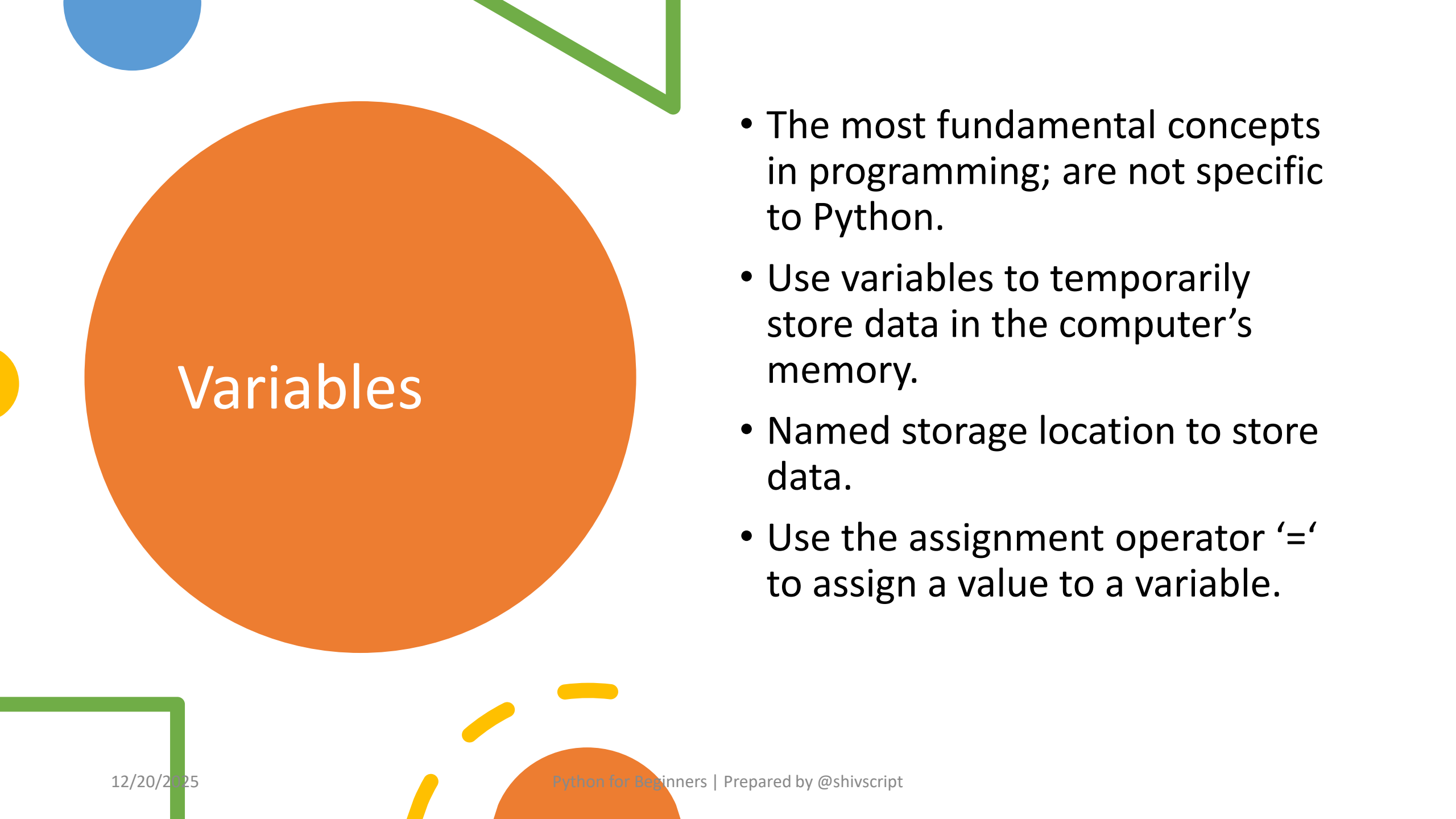
Python Course

Agenda

- Built-in data types
- Variables
- Getting Input from keyboard
- Type Conversion
- Everything you need to know about “Strings”

Built-in Data Types

- **int:** Integer data type for whole numbers.
- **float:** Floating-point data type for decimal numbers.
- **str:** String data type for text.
- **bool:** Boolean data type for True or False values.
- **list:** Ordered, mutable collection of elements.
- **tuple:** Ordered, immutable collection of elements.
- **Dictionary (dict):** Collection of key-value pairs.
- **set:** Unordered collection of unique elements.



Variables

- The most fundamental concepts in programming; are not specific to Python.
- Use variables to temporarily store data in the computer's memory.
- Named storage location to store data.
- Use the assignment operator '=' to assign a value to a variable.



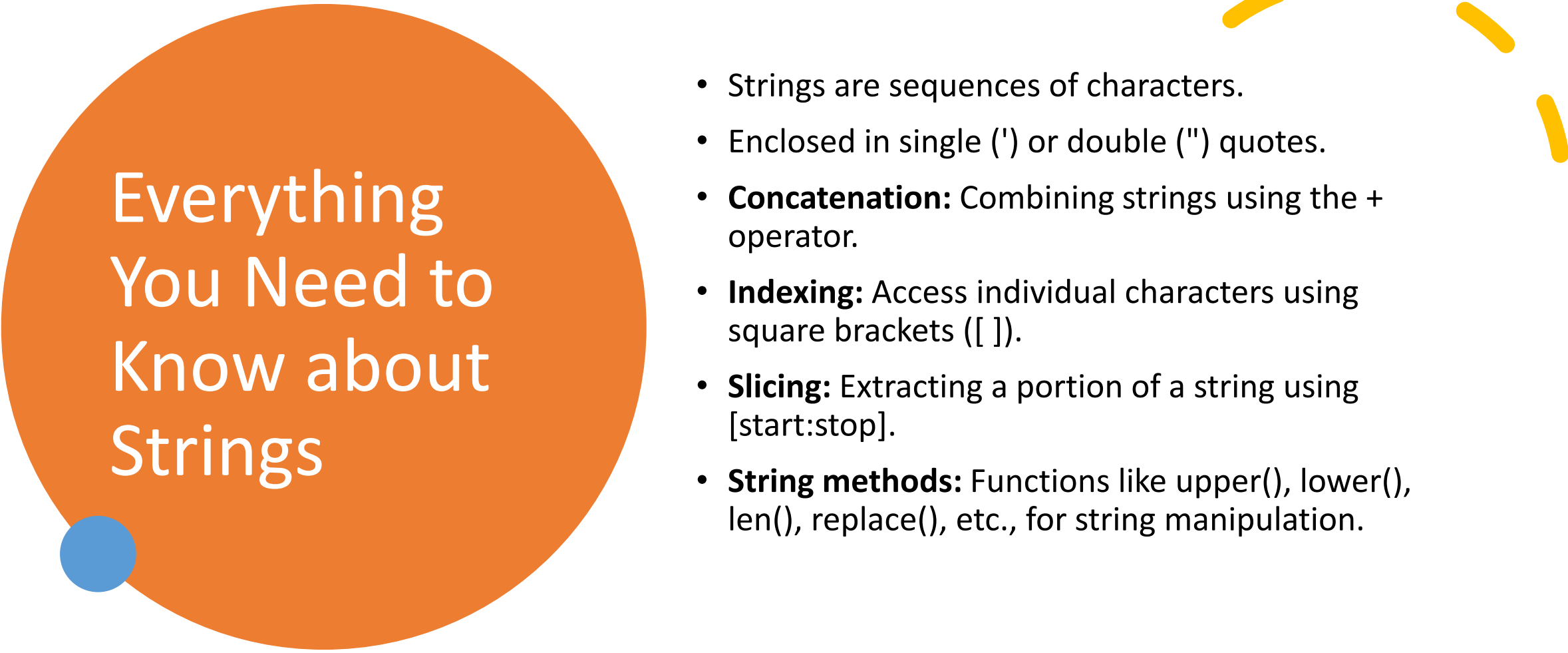
Getting Input from Keyboard

- Use the `input()` function to get input from the keyboard.
- Example: `name = input("Enter your name: ")`

Type Conversion

Use functions like `int()`, `float()`, `str()`, etc., to convert between data types.

Example: `num_str = "10",
num = int(num_str)`



Everything You Need to Know about Strings

- Strings are sequences of characters.
- Enclosed in single (') or double (") quotes.
- **Concatenation:** Combining strings using the + operator.
- **Indexing:** Access individual characters using square brackets ([]).
- **Slicing:** Extracting a portion of a string using [start:stop].
- **String methods:** Functions like upper(), lower(), len(), replace(), etc., for string manipulation.

Questions

Variables and Input:

1. Create a variable **my_age** and assign your age to it. Print a message using this variable.
2. Ask the user to enter their favorite food and print a message incorporating this input.

Type Conversion:

1. Convert the string **"42"** to an integer and print the result.
2. Convert the floating-point number **3.14159** to a string and print the result.

Strings:

1. Concatenate the strings "Hello" and "World!" and print the result.
2. Use string indexing to extract the third character from the string "Python".
3. Take a sentence as input and print only the first five words.

Day-3

Python Course

12/20/2025

Python for Beginners | Prepared by @shivscript

```
self.file = None
self.fingerprints = set()
self.logdups = True
self.debug = debug
self.logger = logging.getLogger(__name__)
if path:
    self.file = open(os.path.join(path, 'requests.log'), 'a')
    self.file.seek(0)
    self.fingerprints.update([request])

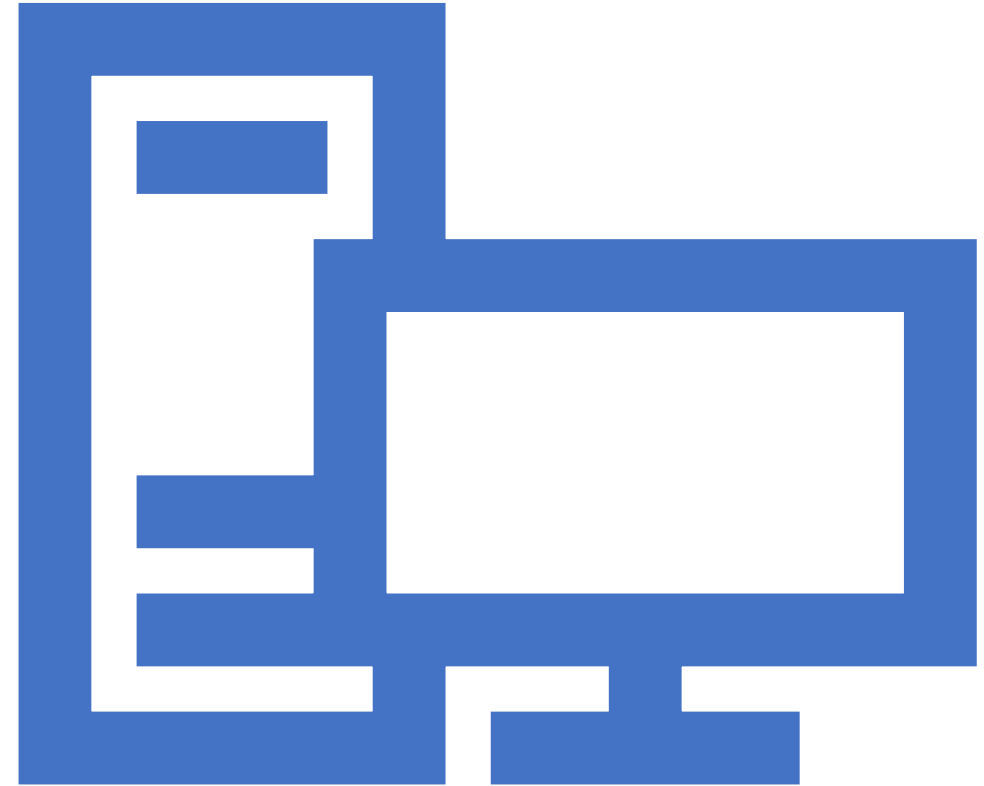
@classmethod
def from_settings(cls, settings):
    debug = settings.getbool('SUPERFILTER_DEBUG')
    return cls(job_dir(settings), debug)

def request_seen(self, request):
    fp = self.request_fingerprint(request)
    if fp in self.fingerprints:
        return True
    self.fingerprints.add(fp)
    if self.file:
        self.file.write(fp + os.linesep)

def request_fingerprint(self, request):
    return request_fingerprint(request)
```

Agenda

- Arithmetic Operations > Operator Precedence
- If Statements > Logical Operators, Comparison Operators
- While Loop
- For Loop



Arithmetic Operations

- Arithmetic operations involve basic mathematical calculations like addition, subtraction, multiplication, division, modulus (remainder after division), and exponentiation (raising a number to a power).

Operator	Meaning	Example
+	Addition	$4 + 7 \longrightarrow 11$
-	Subtraction	$12 - 5 \longrightarrow 7$
*	Multiplication	$6 * 6 \longrightarrow 36$
/	Division	$30 / 5 \longrightarrow 6$
%	Modulus	$10 \% 4 \longrightarrow 2$
//	Quotient	$18 // 5 \longrightarrow 3$
**	Exponent	$3 ** 5 \longrightarrow 243$

Operator Precedence

- Operator precedence determines the order in which operations are performed. Some operators have higher precedence than others, and parentheses can be used to override the default precedence.

Precedence Level	Operator	Explanation
1 (highest)	()	Parentheses
2	**	Exponentiation
3	-a, +a	Negative, positive argument
4	*, /, //, %, @	Multiplication, division, floor division, modulus, at
5	+, -	Addition, subtraction
6	<, <=, >, >=, ==, !=	Less than, less than or equal, greater, greater or equal, equal, not equal
7	not	Boolean Not
8	and	Boolean And
9	or	Boolean Or

Example

- $12 // 4 + 2 ** 4 - 5$
- Predict the result of $3 + 5 * 2 // 4$
- Predict the result of $3 + 5 // 2 * 4$
- $7 + (6 \times 5^2 + 3)$
- $3 + (4 - (11 // 5)) ** 2 \% 2$

If Statements

- If statements allow you to make decisions in your code. They check a condition and execute a block of code if the condition is true; otherwise, an optional else block is executed.
- Three forms of if...else statement.
 - If statement
 - If...else statement
 - If...elif...else statement

if condition:
Statement

elif condition:
statement

else:
statement

Logical and Comparison Operators

- Logical operators (AND, OR, NOT) are used to combine or negate conditions in if statements. Comparison operators (<, >, ==, !=, etc.) are used to compare values.

python

 Copy code

```
num = 25
if num > 10 and num < 30:
    print("Number is between 10 and 30.")
if num < 0 or num % 2 == 0:
    print("Number is negative or even.")
```

Practice Question

- Write an if statement to check if a number is positive, negative, or zero.
- Use logical operators to check if a user-entered number is both greater than 5 and less than 10.
- Write a program to check whether the input number is even or odd.

While Loop

- While loops repeatedly execute a block of code as long as a specified condition is true.

**WHILE LOOP
IN PYTHON**



Practice Question

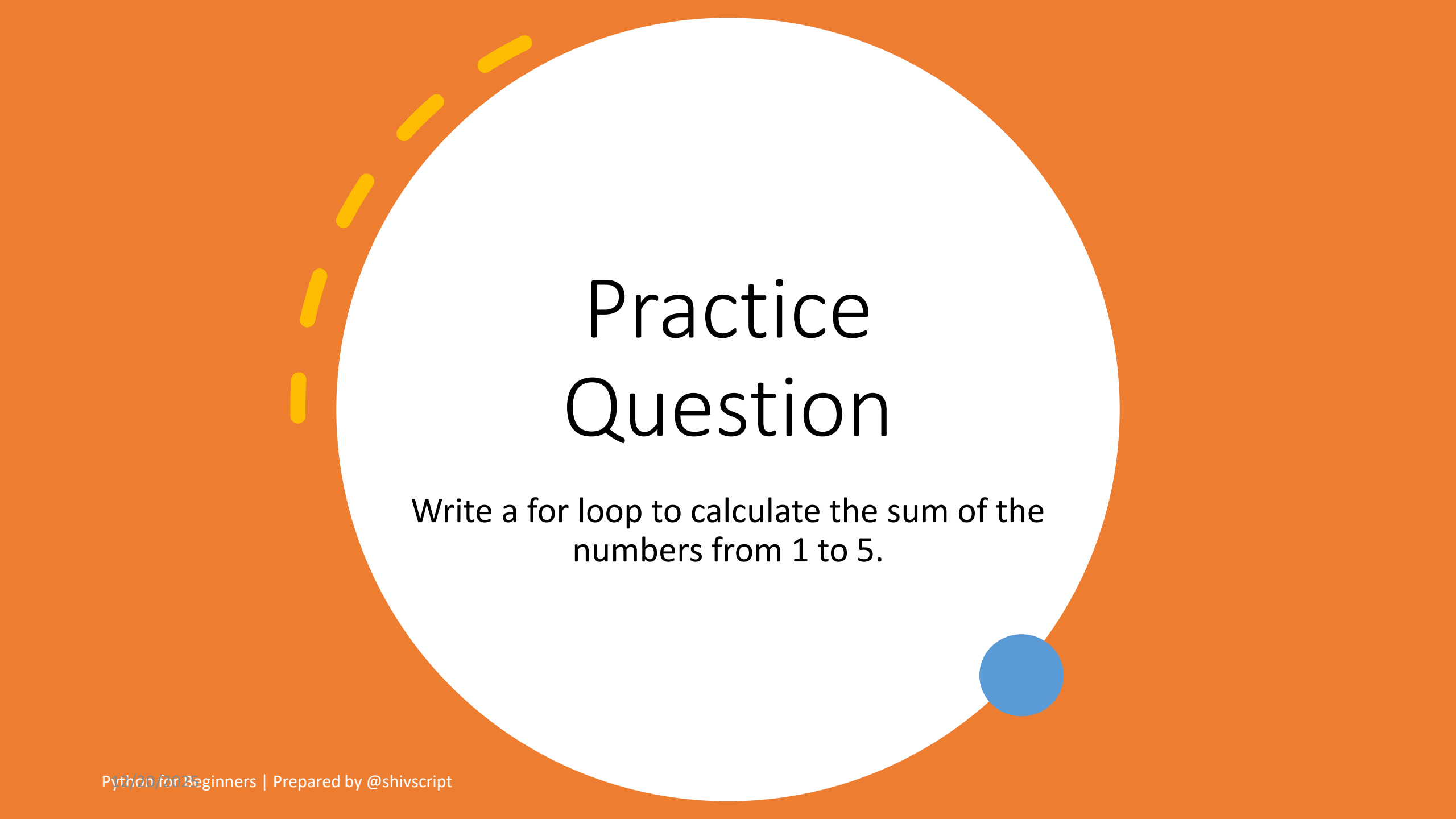
- Write a while loop to print the count of numbers from 1 to 6.
- Write a while loop to print the first 3 multiples of 4.

For Loop

- For loops are used to iterate over a sequence (such as a list, tuple, or string) and execute a block of code for each element in the sequence.



for-loop
in
Python



Practice Question

Write a for loop to calculate the sum of the numbers from 1 to 5.

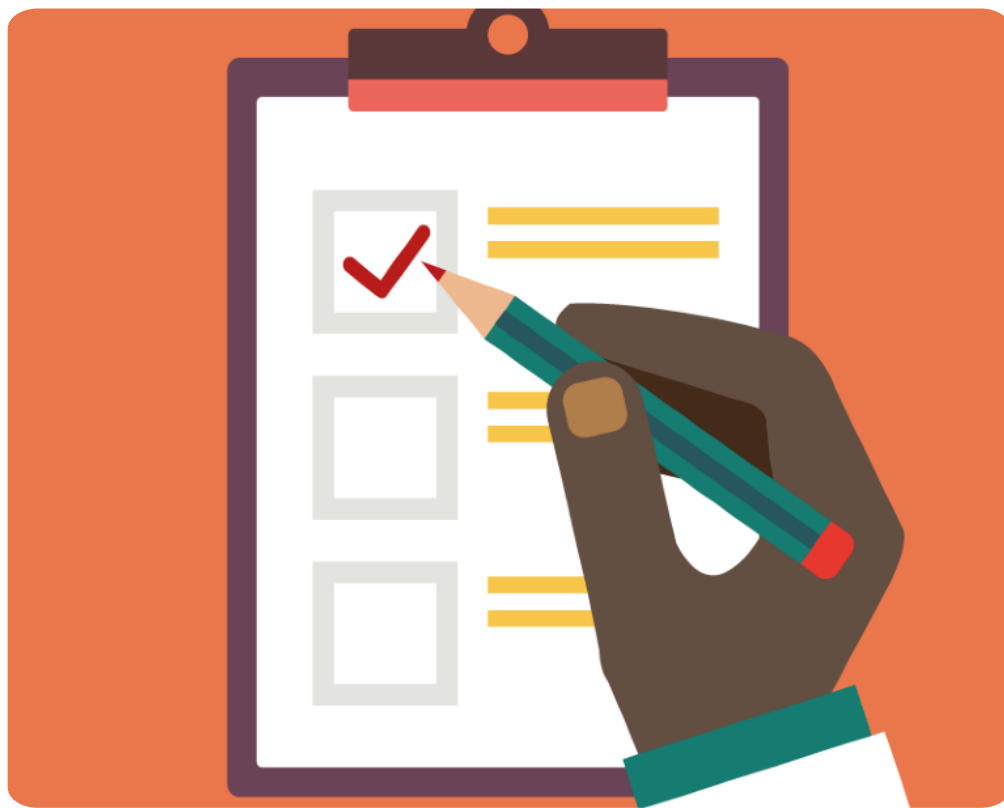
Day-4 & 5

Python Course

12/20/2025

Python for Beginners | Prepared by @shivscript





Agenda

- List
- Tuple
- Dictionary



List

- A list is a versatile and ordered collection of elements in Python. It can contain various data types such as numbers, strings, or other lists.
- Characteristics:
 - Created using square brackets [].
 - Elements are indexed starting from 0.
 - **Mutable:** Elements can be modified, added, or removed after creation.

Python List Methods

Input	Method	Output
	<code>.append()</code>	
	<code>.insert(1, )</code>	
	<code>.pop(1)</code>	
	<code>.remove()</code>	
	<code>.reverse()</code>	
	<code>.sort()</code>	
	<code>.index()</code>	2
	<code>.count()</code>	2

Tuple

- A tuple is an ordered and immutable collection of elements in Python. Once created, the elements cannot be changed or added.
- Characteristics:
 - Created using parentheses ().
 - Elements are indexed starting from 0.
 - **Immutable:** Once a tuple is created, its elements cannot be modified.

Python Tuple Methods

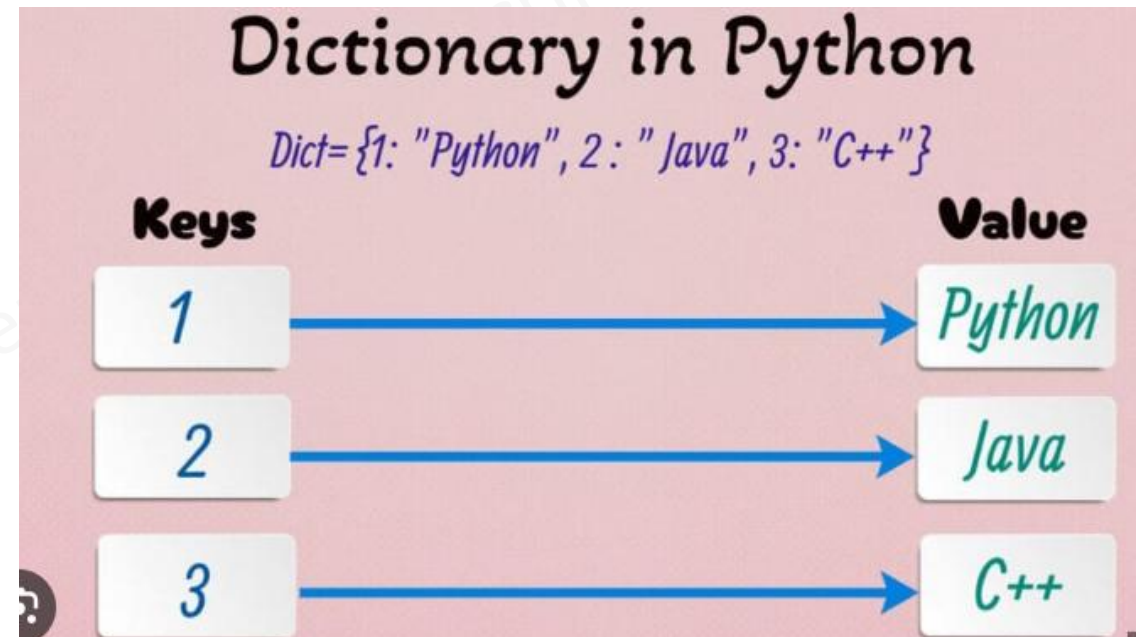
Python has two built-in methods that you can use on tuples.

Method	Description
<u>count()</u>	Returns the number of times a specified value occurs in a tuple
<u>index()</u>	Searches the tuple for a specified value and returns the position of where it was found

Dictionary

A collection used to store data values in key:value pairs.

- Characteristics:
 - Ordered* (from Python version 3.7 onward)
 - Changeable
 - Does not allow duplicates
- Syntax:
 - Dictionaries are written with curly brackets {}.
 - Example: `my_dict = {'key1': 'value1', 'key2': 'value2'}`



Python Dictionary Methods

Python has a set of built-in methods that you can use on dictionaries.

Method	Description
<u>clear()</u>	Removes all the elements from the dictionary
<u>copy()</u>	Returns a copy of the dictionary
<u>fromkeys()</u>	Returns a dictionary with the specified keys and value
<u>get()</u>	Returns the value of the specified key
<u>items()</u>	Returns a list containing a tuple for each key value pair
<u>keys()</u>	Returns a list containing the dictionary's keys
<u>pop()</u>	Removes the element with the specified key
<u>popitem()</u>	Removes the last inserted key-value pair
<u>setdefault()</u>	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
<u>update()</u>	Updates the dictionary with the specified key-value pairs
<u>values()</u>	Returns a list of all the values in the dictionary



Day-6

Python Course

Agenda

- Built-in Functions
 - Type Conversion Functions
 - Math Functions
 - Sequence Functions
 - Input/Output Functions, etc.

Type Conversion Functions

`int(x)`: Converts `x` to an integer.

`float(x)`: Converts `x` to a floating-point number.

`str(x)`: Converts `x` to a string.

`list(x)`: Converts `x` to a list.

`tuple(x)`: Converts `x` to a tuple.

Math Functions

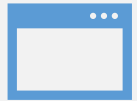
`abs(x)`: Returns the absolute value of x.

`round(x)`: Rounds x to the nearest integer.

`max(iterable)`: Returns the maximum value in an iterable.

`min(iterable)`: Returns the minimum value in an iterable.

Sequence Functions



`len(iterable)`: Returns the length of an iterable (e.g., string, list).



`sum(iterable)`: Returns the sum of elements in an iterable.

Input/Output Functions



`print()`: Prints objects to the console.



`input(prompt)`: Reads a line from the console.

Boolean and Type Inspection Functions

`bool(x)`: Converts `x` to a Boolean value.

`type(x)`: Returns the type of `x`.

`isinstance(x, type)`: Checks if `x` is an instance of a specified type.

Iterable Functions



`range(start, stop, step)`: Generates a sequence of numbers.



`enumerate(iterable)`: Returns pairs of index and value.



`zip(iterable1, iterable2, ...)`: Combines elements from multiple iterables into tuples.

String Functions



`len(string)`: Returns the length of a string.



`str.upper()`, `str.lower()`: Converts a string to uppercase or lowercase.



`str.split(separator)`: Splits a string into a list of substrings.

Miscellaneous Functions



`len(object)`



`input(prompt)`



`id(object)`



`dir(object)`



`help(object)`



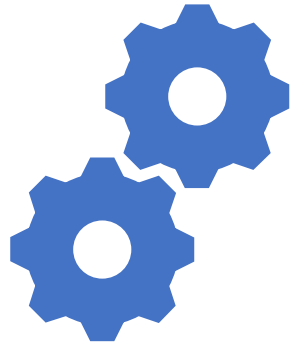
Day-7, 8

Python Course

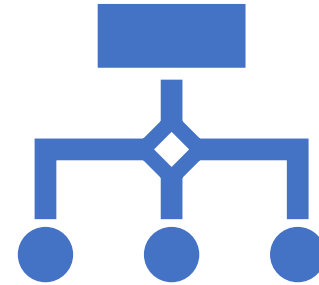
Agenda

- Functions in Python
- Functions with & without return statements
- Parameters and Arguments
- Default Parameters
- Scope of Variables

What is a function? | Why to use functions?



A function is a reusable block of code that performs a specific task.



Functions help with code organization, reusability, and readability.

Anatomy of a Function

python

 Copy code

```
def function_name(parameters):  
    # Function body  
    # Code to perform a specific task  
    return result
```


Defining a function

python

 Copy code

```
def greet(name):  
    return f"Hello, {name}!"
```

python

 Copy code

```
result = greet("Alice")
```

Calling a function

python

Copy code

```
def add_numbers(a, b):  
    # 'a' and 'b' are parameters  
    return a + b
```

python

Copy code

```
result = add_numbers(3, 5)
```

Parameters and Arguments in Function

- Parameters are the variables listed inside the parentheses in a function definition.
- Arguments are the actual values passed to a function when it is called.

Function without return

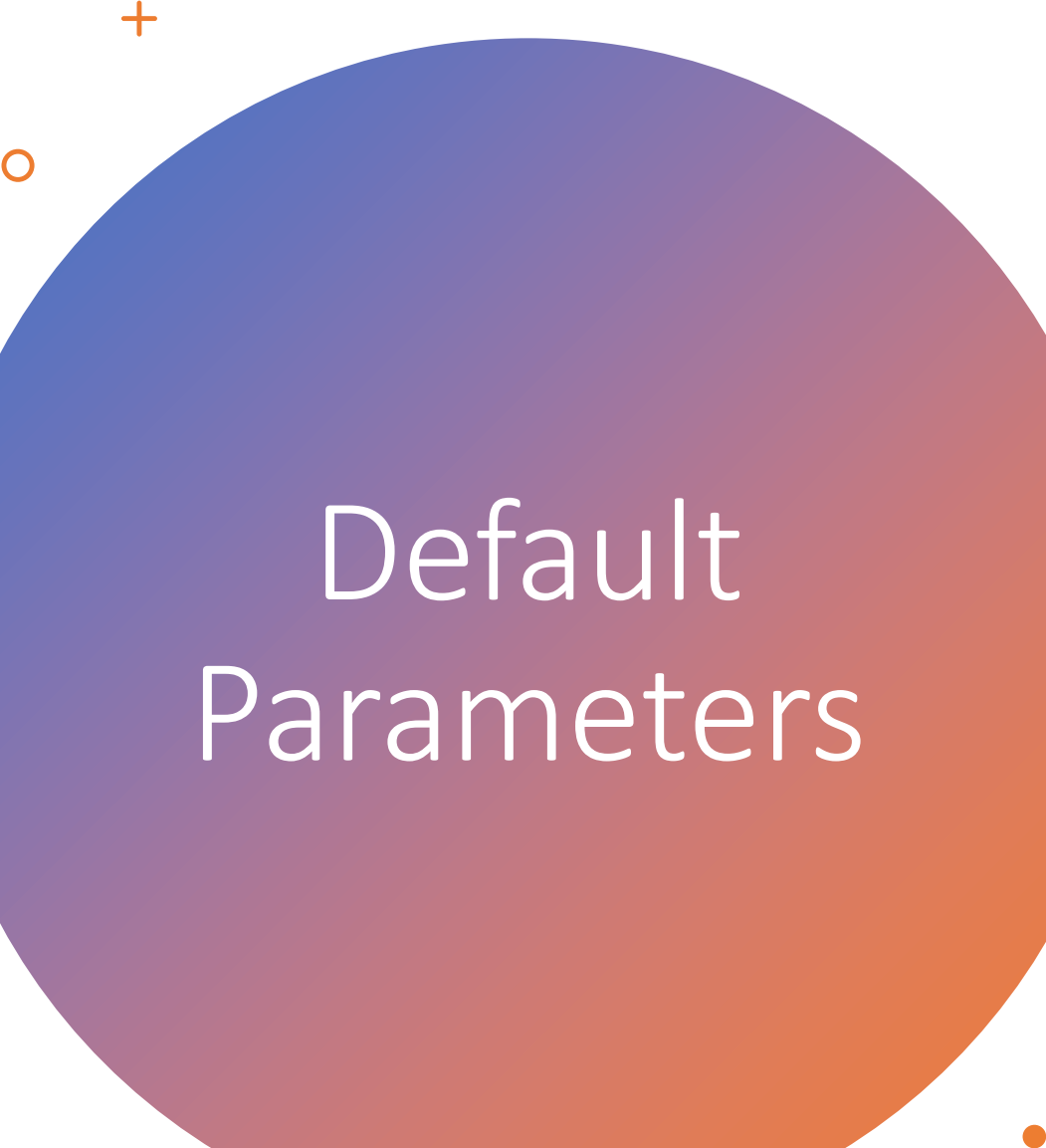
- A function without a return statement implicitly returns None.

Note that while this function doesn't use return to provide a specific value, it's still valid and common for functions to perform actions (like printing to the console) without returning a value.

python

 Copy code

```
def greet(name):  
    print(f"Hello, {name}!")  
  
# Calling the function  
greet("Alice")
```



Default Parameters

- Default parameters allow you to specify a default value for a function parameter. The default value is used if the caller doesn't provide a value for that parameter.
- Default parameters make functions more flexible and allow you to define functions with a set of default behaviors while still allowing customization.

python

 Copy code

```
def function_name(parameter1, parameter2=default_value):  
    # Function body  
    # Code using parameter1 and parameter2  
    return result
```

Defining default parameter

- “parameter1” is a regular parameter that must be provided by the caller.
- “parameter2” is a parameter with a default value (default_value), meaning it is optional. If the caller doesn't provide a value for parameter2, the default value is used.

Practice Questions

python

 Copy code

```
def display_info(name, age=25, city="Unknown"):
    return f"{name} is {age} years old and lives in {city}."
```

Scope of variables



- **Definition:** Variable scope refers to the region of the code where a variable can be accessed or modified.
- **Importance in Python Programming:** Understanding variable scope is crucial for writing maintainable and bug-free code.

Local Scope

- Variables defined within a function.

python

 Copy code

```
def example_function():  
    local_variable = "I am local"
```

Accessing Local Variables

- Local variables are accessible only within the function they are defined.
- Attempting to access them outside the function results in an error.

python

 Copy code

```
def example_function():  
    local_variable = "I am local"  
  
print(local_variable) # Raises an error
```

Global Scope

- Variables defined outside of any function.

python

 Copy code

```
global_variable = "I am global"
```

Accessing Global Variables

- Global variables can be accessed and modified from any part of the code.

Example

python

 Copy code

```
global_variable = "I am global"

def example_function():
    print(global_variable)
```

Enclosing (Nonlocal) Scope

- Variables in nested functions that are not local nor global.
- Nonlocal variables are used in nested functions to reference and modify variables from the outer (enclosing) function's scope.

```
python Copy code  
  
def outer_function():  
    outer_variable = "I am nonlocal"  
  
    def inner_function():  
        nonlocal outer_variable  
        outer_variable = "I am modified"  
        print(outer_variable)  
  
    inner_function()  
    print(outer_variable)  
  
outer_function()
```

Best Practices

- Minimize the use of global variables.
- Use local variables whenever possible to avoid unintended side effects.
- Understanding variable scope is crucial for writing maintainable and bug-free code.





Day-9

Python Course

Agenda

- Keywords and Identifiers
- Case sensitivity
- Nested Structures



Identifiers

- Identifiers are names given to entities in Python, such as variables, functions, classes, etc.
- Rules for Identifiers:
 - Must start with a letter (a-z, A-Z) or an underscore (_)
 - Can be followed by letters, underscores, or digits (0-9)
 - Cannot be a Python keyword

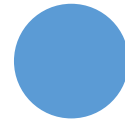
python

 Copy code

```
variable_name = 10
function_name()
class ClassName:
    pass
```

Keywords

- Keywords are reserved words in Python, which have a specific meaning and cannot be used as identifiers.
- List of Common Keywords:
 - if, else, while, for, def, class, True, False, None, etc.



```
# Python program that prints the complete list of keywords

import keyword
print(keyword.kwlist)

['False', 'None', 'True', 'and', 'as', 'assert', 'async',
'await', 'break', 'class', 'continue', 'def', 'del',
'elif', 'else', 'except', 'finally', 'for', 'from',
'global', 'if', 'import', 'in', 'is', 'lambda',
'nonlocal', 'not', 'or', 'pass', 'raise', 'return', 'try',
'while', 'with', 'yield']
```

Case sensitivity

- Python is case-sensitive.
- `myVariable` and `myvariable` are treated as different identifiers.
- Common mistakes:
 - Avoid using Python keywords as identifiers.
 - Be mindful of case sensitivity.
- Best practices:
 - Choose meaningful and descriptive names for identifiers.
 - Follow PEP 8 style guide for naming conventions.



Nested 'if' statements

- **Definition:** If statements inside other if statements
- **Example:** Checking conditions within conditions

python

 Copy code

```
if condition1:  
    if condition2:  
        # Code block
```

Nested 'for' loops

python

 Copy code

```
for i in range(5):  
    for j in range(3):  
        # Code block
```

- **Definition:** Loops inside other loops
- **Example:** Iterating through multiple levels

Nested functions

- **Definition:** Functions calling other functions
- **Example:** Encapsulating logic in modular pieces

python

 Copy code

```
def outer_function():  
    # Code block  
    inner_function()  
  
def inner_function():  
    # Code block
```

Benefits of Nested Structures

- **Modularity:** Breaking down complex problems into manageable parts
- **Readability:** Organizing code for easier understanding
- **Reusability:** Functions can be reused in different contexts



Best Practices

- **Keep it simple:** Avoid excessive nesting for clarity
- **Commenting:** Add comments to explain nested structures
- **Testing:** Verify each level of nesting to ensure correct behavior





Day-10 & 11

Python Course

Agenda

- Comments
- Flow control in Python: 'pass', 'break', and 'continue' statements



Comments in Python

- Comments are used to explain code for better understanding.
- It is a tool for documentation.
- Importance: code readability and documentation.
- Comments clarify code, provide context, and help others understand your intentions
- Two types of comments:
 - Single-line comments
 - Multiple-line comments

python

 Copy code

```
# This is a single-line comment
```

```
"""
```

```
This is a  
multi-line comment
```

```
"""
```

'pass' statement

- 'pass' is a null operation; nothing happens when it executes
- Use case: Placeholder in situations where syntax requires a statement, but no action is desired.

python

 Copy code

```
if condition:  
    pass # Placeholder, no action
```

'break' statement

- 'break' is used to exit the loop prematurely
- Use case: Terminate a loop based on a certain condition

python

 Copy code

```
for i in range(5):  
    if i == 3:  
        break # Exit the loop when i is 3
```

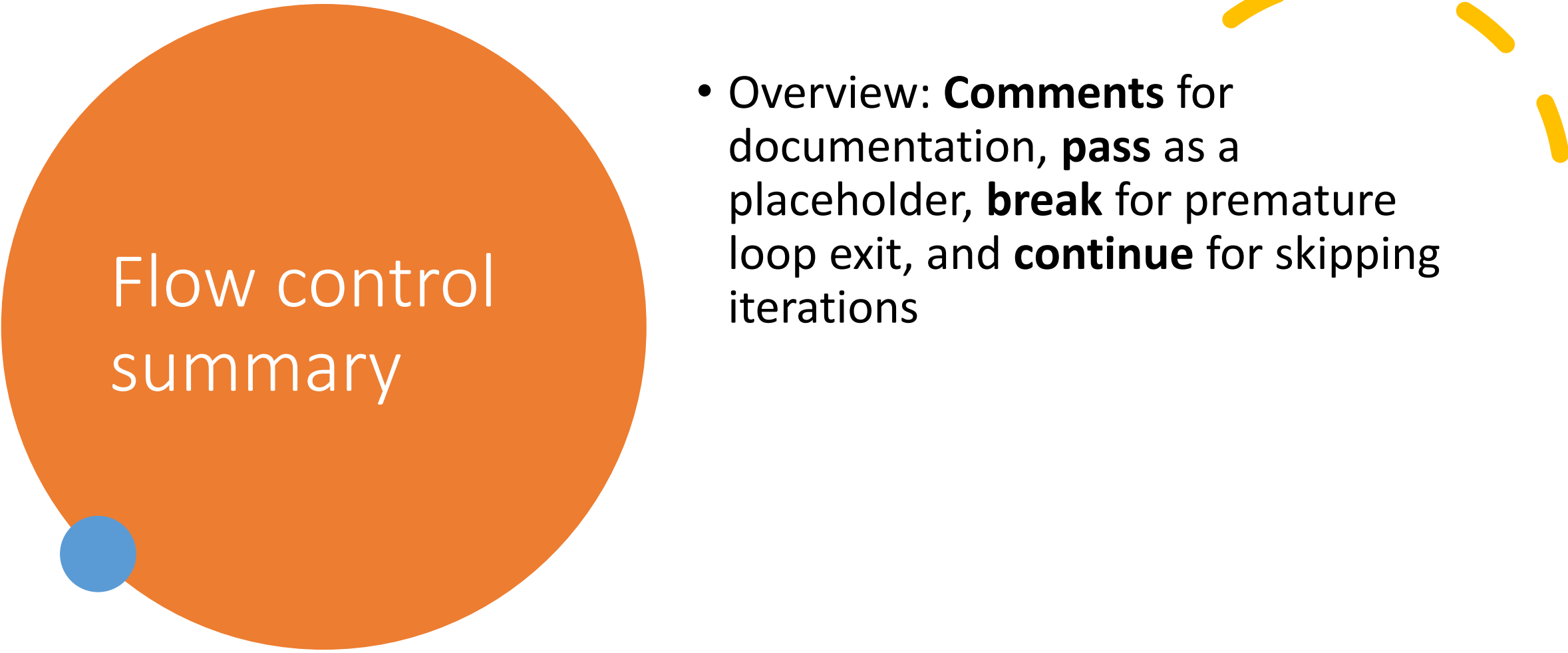
'continue' statement

- 'continue' is used to skip the rest of the code inside a loop for the current iteration
- Use case: Skip specific iterations based on a condition

python

 Copy code

```
for i in range(5):  
    if i == 2:  
        continue # Skip iteration when i is 2  
    # Code that runs for other values of i
```



Flow control summary

- Overview: **Comments** for documentation, **pass** as a placeholder, **break** for premature loop exit, and **continue** for skipping iterations

Best Practices

- Commenting guidelines: Be concise, use clear language
- Use **pass**, **break**, and **continue** judiciously for clean and effective code



Things to remember

- The 'continue' statement is specifically designed for use within loops (such as 'for' and 'while' loops) to skip the rest of the code in the current iteration and move to the next iteration of the loop. It does not have a meaningful use outside of loops.
- If you attempt to use continue outside of a loop, you will encounter a SyntaxError.

python

 Copy code

```
def my_function():  
    print("Before continue")  
    continue  
    print("After continue")  
  
my_function()
```

Practice Questions - pass

python

 Copy code

```
def example_function():  
    for i in range(4):  
        if i == 2:  
            pass  
        print(i)  
  
example_function()
```

python

 Copy code

```
for i in range(5):  
    if i == 3:  
        pass  
    print(i)
```

Practice Questions - continue

python

 Copy code

```
for i in range(3):  
    for j in range(3):  
        if j == 1:  
            continue  
        print(i, j)
```

python

 Copy code

```
for num in range(10):  
    if num % 2 == 0:  
        continue  
    print(num)
```

Practice Questions - break

python

 Copy code

```
numbers = [10, 20, 30, 40, 50]
for num in numbers:
    if num == 30:
        break
    print(num)
```

python

 Copy code

```
for x in range(6):
    print(x)
    if x == 3:
        break
```





Day-12 & 13

Python Course

Agenda

- Python Libraries
 - Built-in
 - Third-party

What are libraries?



Collections of pre-written code that extend Python's functionality

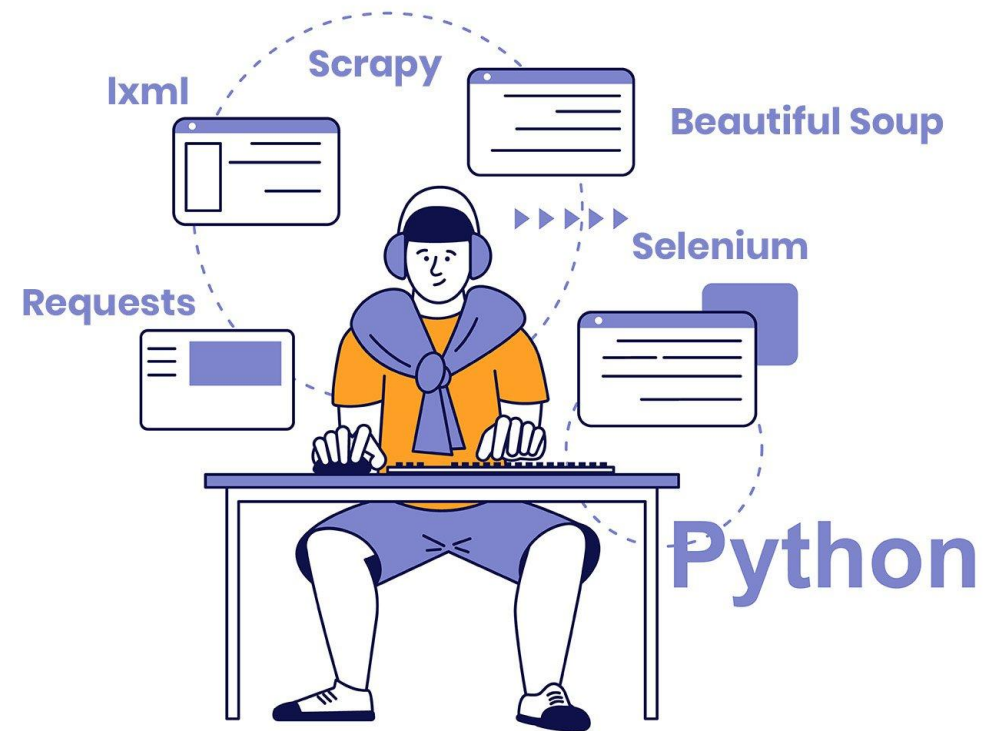


Save time and effort by reusing well-tested code



Offer specialized tools for various tasks

Built-in vs. Third-Party Libraries



Key Built-in Libraries for Beginners

Math: Mathematical functions (e.g., sin, cos, sqrt, pi)

Random: Generating random numbers

Datetime: Working with dates and times

OS: Interacting with the operating system

JSON: Handling JSON data (data exchange format)

Examples

@shivscript

Math: `import math; print(math.sqrt(16))`

Random: `import random; print(random.randint(1, 10))`

Datetime: `import datetime; now = datetime.datetime.now(); print(now)`

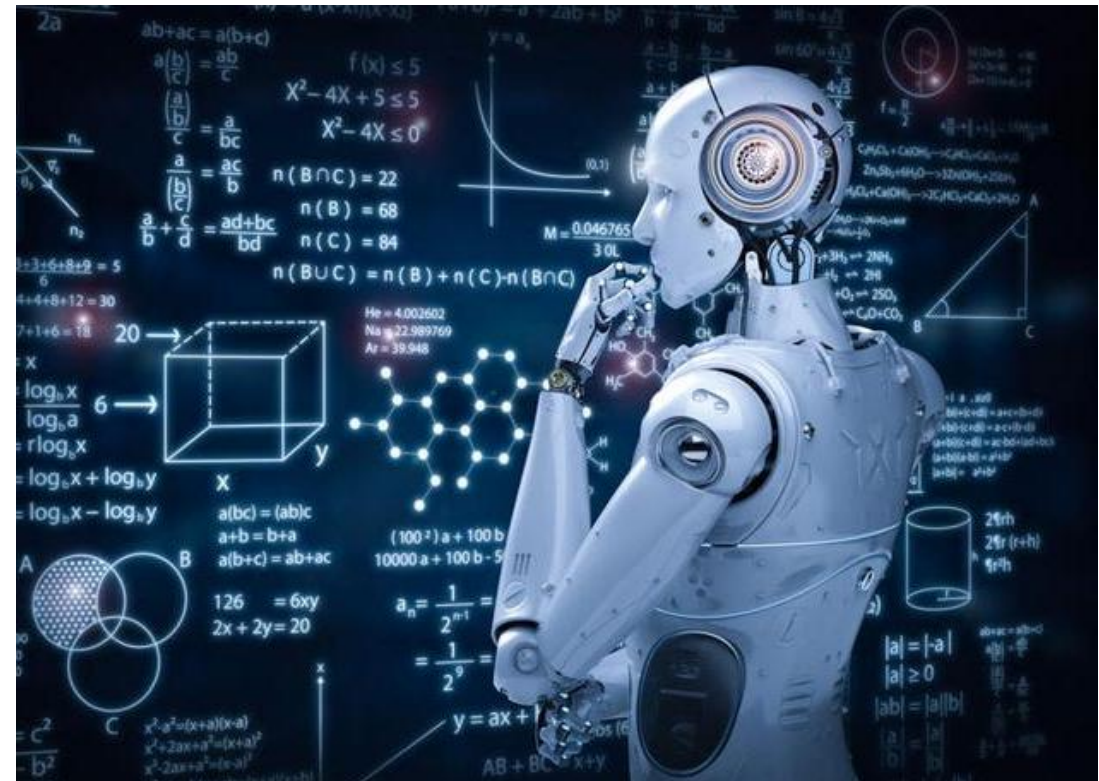
OS: `import os; print(os.getcwd())`

JSON: `import json; data = {"name": "Alice", "age": 30}; json_string = json.dumps(data); print(json_string)`

Exploring More Built-in Libraries



What are Third-Party Libraries?



Installing Third-Party Libraries



shutterstock.com • 398576578

Popular Third-Party Libraries

A thick yellow horizontal bar spans the width of the slide, with a vertical yellow bar extending downwards from its right end.

Exploring More Libraries

A thick yellow horizontal bar spans the width of the slide, with a vertical yellow bar extending downwards from its right end.

Benefits of Using Third-Party Libraries

- Save time and effort: Avoid reinventing the wheel
- Access specialized tools: Solve complex problems efficiently
- Stay up-to-date: Libraries evolve with cutting-edge features
- Build larger, more powerful applications



Responsible Library Usage

- Choose libraries with active development and good documentation
- Understand licensing terms and dependencies
- Update libraries regularly for security and bug fixes



Summary

A thick yellow horizontal bar spans the width of the slide, with a vertical yellow bar extending downwards from its right end.

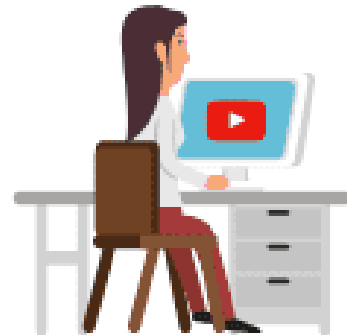


Day-14 &
15

Python Course

Agenda

- Exception Handling and Revision



A girl is watching a video on
Youtube on the computer



Interrupted in watching
video due to internet
disconnectivity suddenly



Stopped
Car punctured
Exception

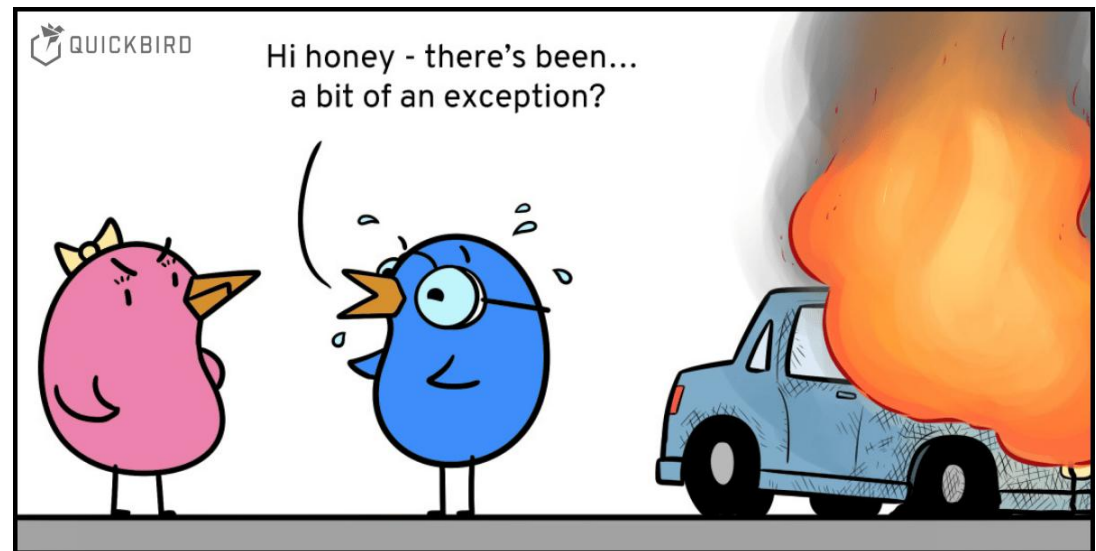
Puncture repaired
Exception Handled

Fig: Realtime Example of Exception Handling

Handling Errors Gracefully with Exception Handling in Python



What are Exceptions?



EXCEPTION!



CALL THE DEVELOPERS!!!!1!

12/20/2025

memes.com

Common Exception Types

List of common Python exceptions:

- ZeroDivisionError
- TypeError
- ValueError
- IndexError
- KeyError
- NameError
- FileNotFoundError
- OSError

The *try* and *except* Blocks

- The ***try block*** contains code that might raise exceptions.
- The ***except blocks*** handle specific exceptions.
- The ***else block*** executes if no exceptions occur.
- The ***finally block*** executes always, even if an exception is raised.

Python

```
try:
    # Code that might raise exceptions
except ExceptionType1:
    # Handle ExceptionType1
except ExceptionType2:
    # Handle ExceptionType2
# ... more except blocks
else:
    # Code to execute if no exceptions occur
finally:
    # Code to execute always, regardless of exceptions
```

Handling Specific Exceptions

- Catch different types of exceptions separately

python

 Copy code

```
try:
    # Code that may raise an exception
except ValueError as ve:
    # Handle ValueError
except FileNotFoundError as fe:
    # Handle FileNotFoundError
```

Zero Division Error

Python

```
try:  
    result = 10 / 0  
except ZeroDivisionError:  
    print("Cannot divide by zero!")
```

Raising Exceptions

- Use the ***raise keyword*** to raise exceptions intentionally.

Python

```
if age < 18:  
    raise ValueError("Age must be 18 or older.")
```

The *finally* Block

- Code within the ***finally*** block runs regardless of whether an exception occurred.
- Ensures code execution, often used for cleanup tasks.

python

 Copy code

```
try:
    # Code that may raise an exception
except ExceptionType as e:
    # Handle the exception
finally:
    # Code that always runs
```

Python

```
try:
    file = open("my_file.txt")
    # Work with the file
finally:
    file.close() # Ensure file is closed even if exceptions occur
```

finally Example

The *else* block

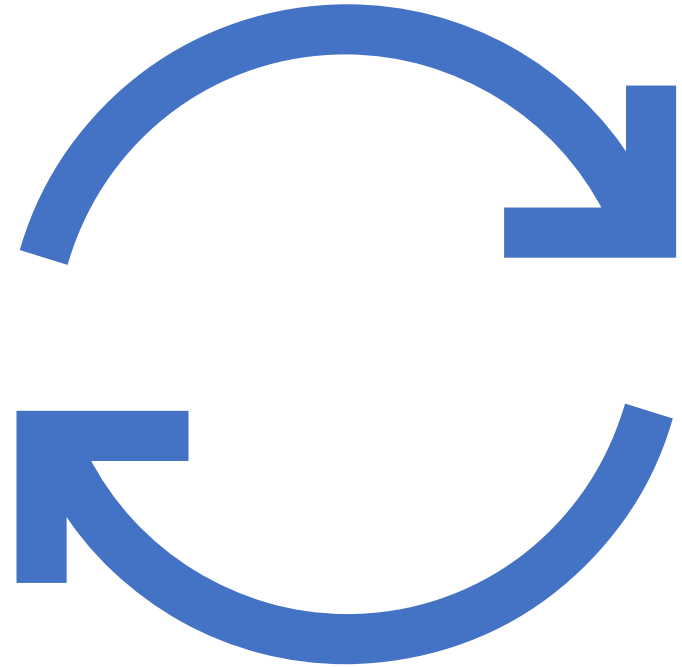
- Executes code only if no exceptions occur within the try block.
- Example: The else block executes only if the input is successfully converted to an integer.

Python

```
try:
    number = int(input("Enter a number: "))
except ValueError:
    print("Invalid input. Please enter a number.")
else:
    print("You entered a valid number:", number)
```


Best Practices

- Handle specific exceptions whenever possible.
- Don't overuse exception handling for normal program flow.
- Provide informative error messages to users.
- Test your exception handling code thoroughly.



Practice Questions for Exception Handling in Python:



1. Write a program that prompts the user for a number and divides it by 2. Handle the `ZeroDivisionError` exception and print a friendly message instead of crashing.



2. Write a program that asks for the user's age. Check if the input is an integer and raise a `ValueError` if not. Print different messages based on age categories (e.g., child, adult, senior).



3. Write a program that performs a calculation involving multiple variables. Use a single `except` block to catch any potential `TypeError` or `ValueError` raised during the calculation. Handle each exception type differently, providing informative messages.



Day-16

Python Course

Agenda

Overview of File I/O in Python

- Interaction with external files
- Reading and writing data
- Fundamental for data processing and storage



Opening a File

python

 Copy code

```
file = open('example.txt', 'r')
```

- **open() Function**
 - Syntax: *open(file, mode)*
 - Modes:
 - 'r': Read (default)
 - 'w': Write
 - 'a': Append
 - 'b': Binary mode
 - 'x': Exclusive creation

Reading from a File

- Reading Methods
 - `read(size)`: Reads entire contents
 - `readline()`: Reads a single line
 - `readlines()`: Reads all lines into a list

python

```
with open('example.txt', 'r') as file:  
    content = file.read()
```

Writing to a File

- Writing Methods
 - `write(string)`: Writes a string to the file
 - `writelines(lines)`: Writes a list of lines to the file

```
python
```

```
with open('example.txt', 'w') as file:  
    file.write("Hello, File!")
```


Appending to a File

- Appending to an Existing File
 - Use mode 'a'
 - Appends content to the end of the file

```
python
```

```
with open('example.txt', 'a') as file:  
    file.write("\nAppending additional content.")
```

Closing a File

- close() Method
 - Manually close the file after operations
 - Important for resource management

python

```
file = open('example.txt', 'r')  
content = file.read()  
file.close()
```

Handling Exceptions

- Exception Handling for File I/O
 - Handle FileNotFoundError, PermissionError, etc.
 - Prevents crashes and provides user-friendly messages

```
python
```

```
try:  
    with open('example.txt', 'r') as file:  
        content = file.read()  
except FileNotFoundError:  
    print("File not found.")
```

Using “with” Statement

- Automatically closes the file when the block is exited
- Context management for clean and efficient code

python

```
with open('example.txt', 'r') as file:  
    content = file.read()
```

Binary File Operations

- Binary Mode ('b')
 - Read/Write binary data
 - Useful for images, audio, etc.

```
python
```

```
with open('image.jpg', 'rb') as binary_file:  
    image_data = binary_file.read()
```

Conclusion

- Summary of Key Points
 - Open, read, write, and close files
 - Use with statement for automatic closing
 - Handle exceptions for robust file operations
 - Explore binary mode for non-textual data



Python Course

Day-17, 18,
19, 20, 21



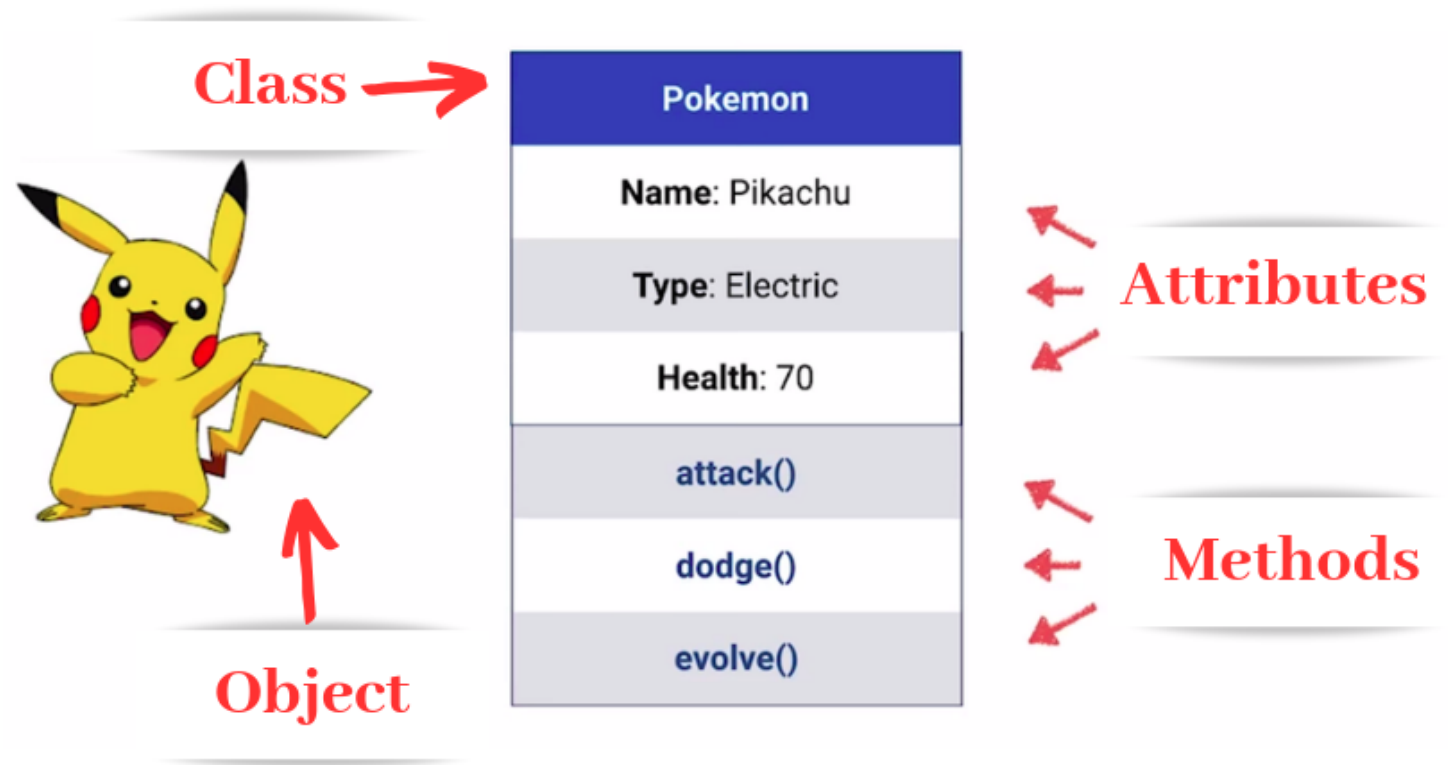


Agenda

- Object-Oriented Programming in Python (OOP)

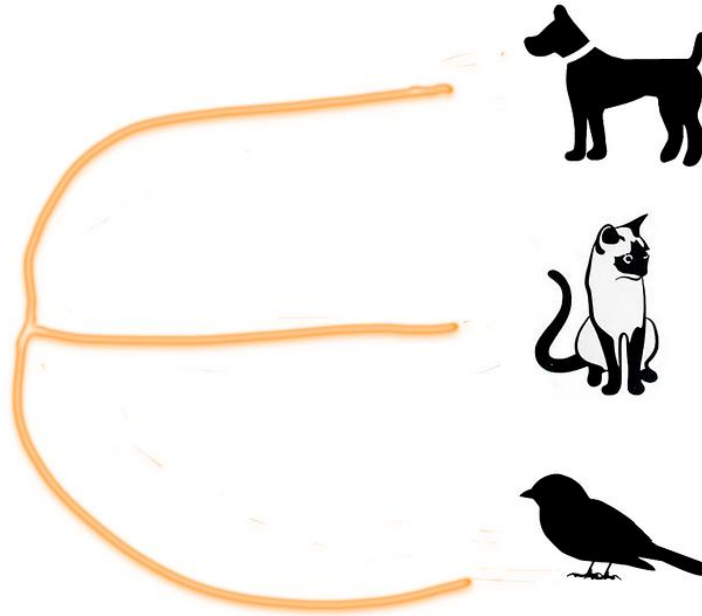
What is OOP?

- Organizing code like real-world things (cars, trees, users).
- Imagine objects with properties (color, age, etc.) and actions (drive, grow, login).
- OOP is a way of writing code by organizing it into reusable structures called "objects" that contain both data and functionality.



CLASSES AND OBJECTS

ANIMALS



Age: 20

Name: Dog

Age: 10

Name: Cat

Age: 5

Name: Bird

Benefits of OOP

Code reusability

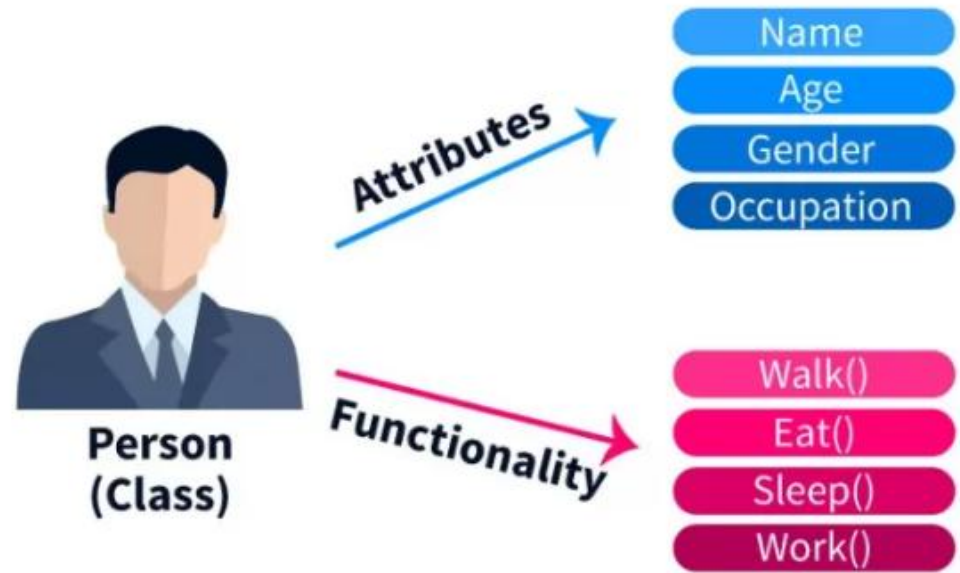
Modularity

Maintainability

Encapsulation of data

Classes and Objects

What is Class?





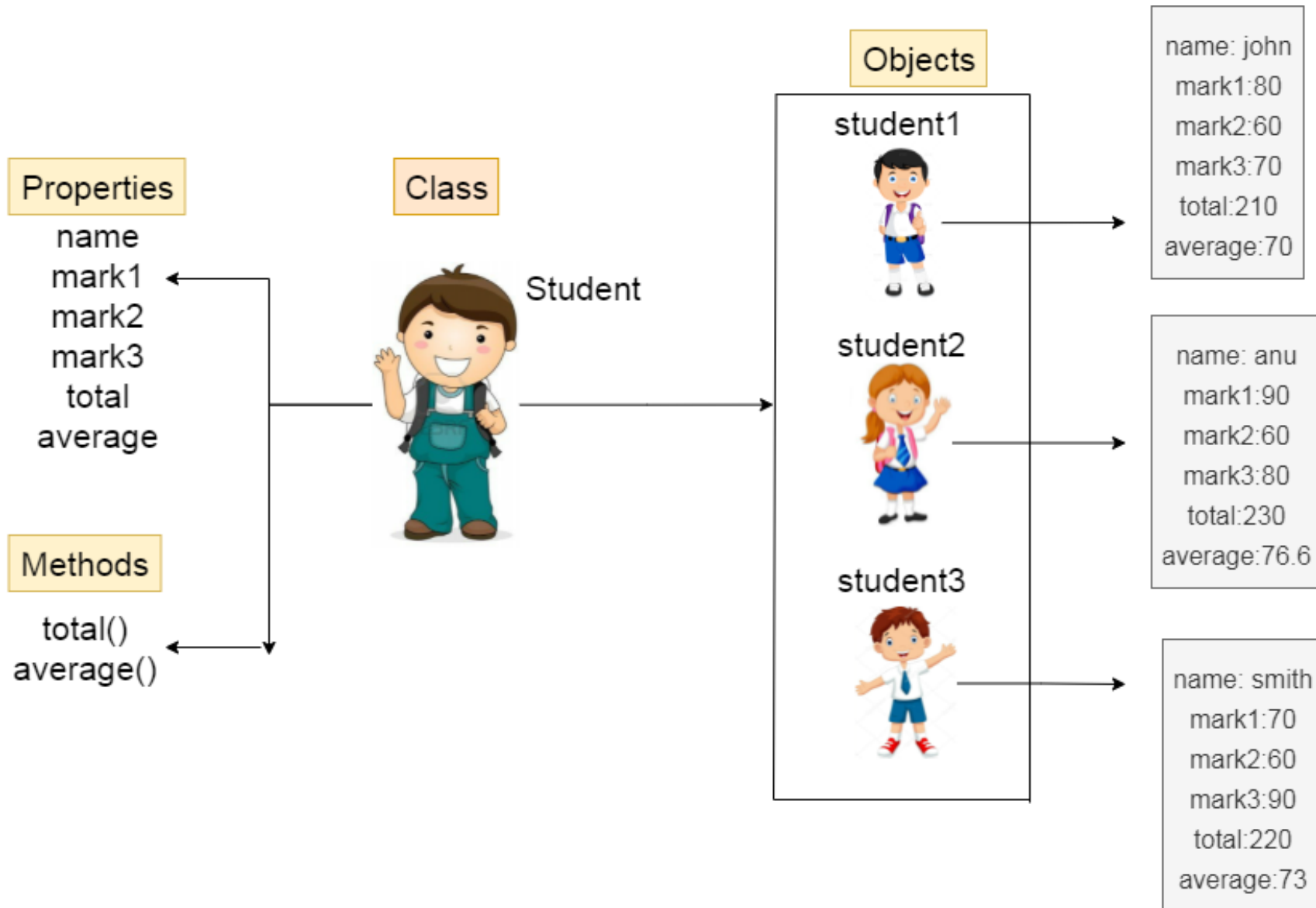
**Person
(class)**

Attributes

- Name
- Age
- Gender
- Occupation

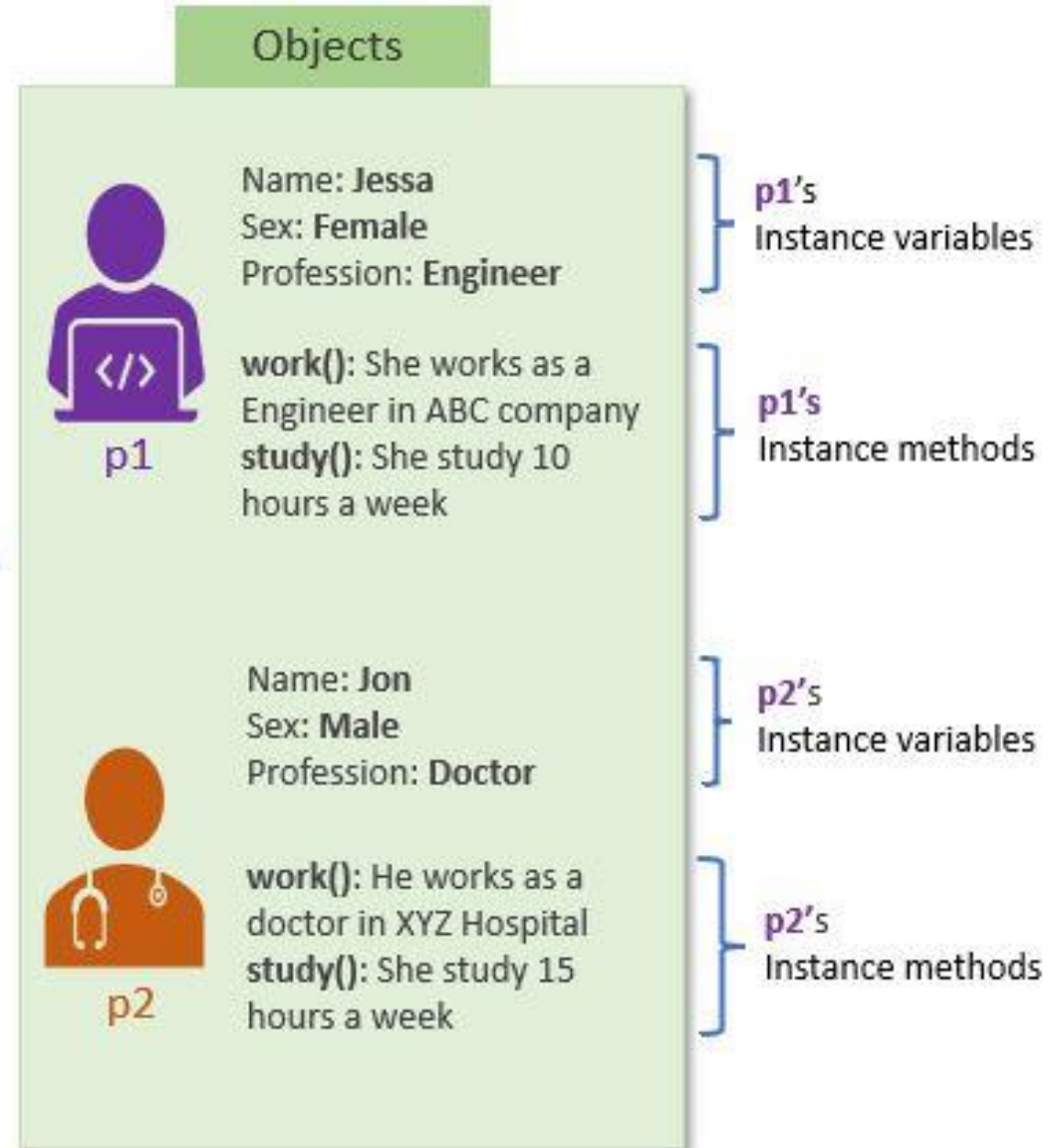
Functionality

- Walk ()
- Eat ()
- Sleep ()
- Work ()





Blueprint to create objects





```
class Dog:
    def __init__(self, name, breed):
        self.name = name
        self.breed = breed

    def bark(self):
        print("Woof!")

my_dog = Dog("Fido", "Labrador")
print(my_dog.name) # Output: Fido
my_dog.bark()      # Output: Woof!
```


__init__ (Constructor)

A constructor in Python is a special type of method which is used to initialize the instance members of the class.

```
class class_name:  
    def __init__(self):  
        #default Constructor  
  
    def __init__(self,a,b,c):  
        #parameterized Constructor
```



self (Reference to the Instance)



```
class human():  
    def __init__(self, age=0, sex="?"):  
        self.age = age  
        self.sex = sex  
    def speak(self):  
        print("Hello i am :", self.age, "and ", self.sex)
```

```
man = human()
```

```
woman = human()
```



```
class Person:
    def __init__(self, name, age):
        self.name = name # Assigning values to attributes using self
        self.age = age

    def greet(self):
        print("Hello, my name is", self.name) # Using self to access
attributes

person1 = Person("Alice", 30) # Calls __init__ with self referring to
person1
person1.greet() # Calls greet with self referring to person1
```

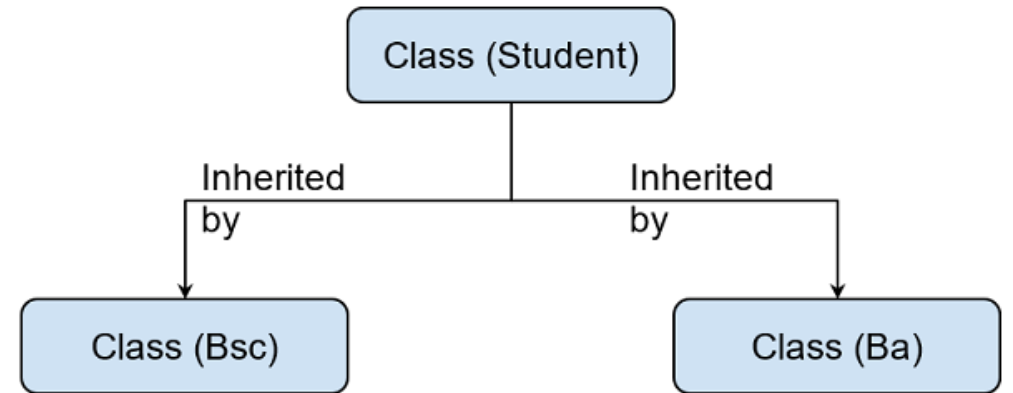
Key Points

- Without `__init__`, objects would start with no attributes or default values.
- `self` allows methods to differentiate between different instances of the class and work with their specific data.
- It's a convention to name the first argument `self`, but any name can technically be used.



Inheritance

- Inheritance allows you to define new classes based on existing ones.
- The new class (subclass) inherits attributes and methods from the existing class (base class).
- This is like extending or specializing the functionality of the base class.



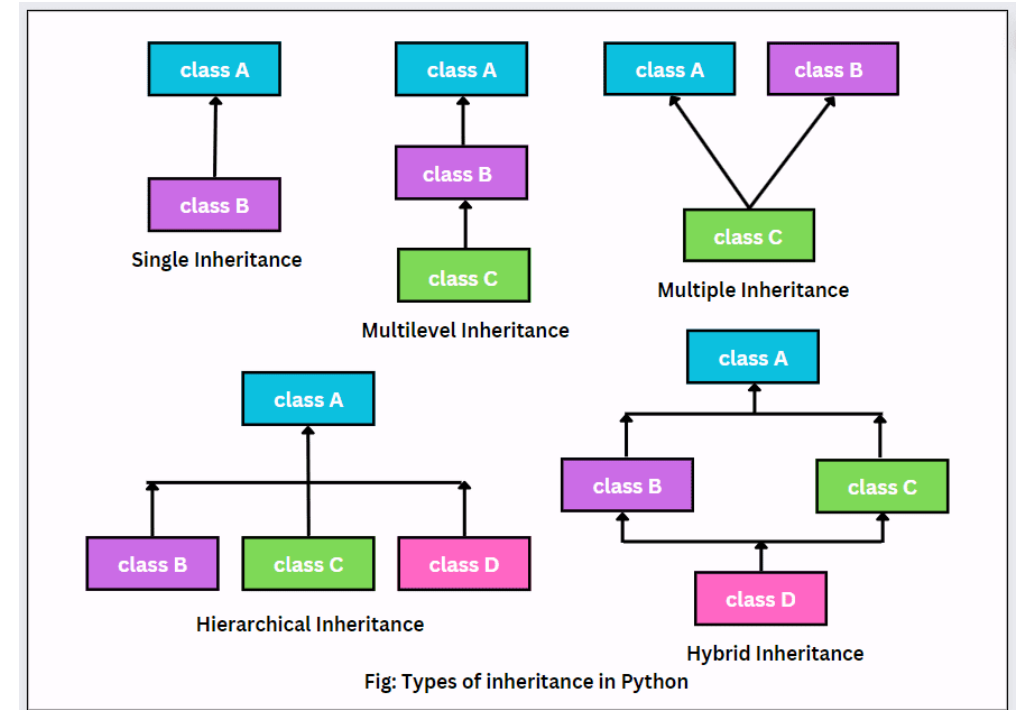
Benefits of Inheritance:

- **Code reusability:** You don't need to write the same code again and again. You can inherit base class functionalities and then add your own specific features.
- **Modularity:** It helps create a hierarchy of classes, making your code more organized and manageable.
- **Reduced complexity:** You can build complex functionality on top of simpler base classes, making your code easier to understand and maintain.



Types of Inheritance

- **Single inheritance:** A subclass inherits from only one base class.
- **Multiple inheritance:** A subclass inherits from multiple base classes (rarely used and can be complex).



Examples:

- **Animal -> Dog, Cat:** `Animal` can define attributes like `species`, `age`, and methods like `breathe`, `move`. `Dog` and `Cat` inherit these but can also have specific attributes and methods like `bark` for `Dog` and `meow` for `Cat`.
- **Shape -> Circle, Square:** `Shape` can define methods like `calculate_area`. `Circle` and `Square` inherit this but have their own formulas for calculating their specific areas.

Method Overriding

- Child classes can override methods from the parent class to provide their own implementation.
- Use ***super()*** within the child class method to access the parent class implementation if needed.

```
class Animal:
    def make_sound(self):
        print("I make a generic animal sound.")

class Dog(Animal):
    def make_sound(self):
        print("Woof!") # Overrides parent class method

class Cat(Animal):
    def make_sound(self):
        super().make_sound() # Calls parent class method then adds its own
        print("Meow!")

dog1 = Dog()
dog1.make_sound()

cat1 = Cat()
cat1.make_sound()
```

Python

```
class Shape:
    def __init__(self, color):
        self.color = color

    def describe(self):
        print(f"I am a shape, and my color is {self.color}.")

class Circle(Shape):
    def __init__(self, color, radius):
        super().__init__(color) # Inherit color from parent class
        self.radius = radius

    def calculate_area(self):
        return 3.14 * self.radius**2

    def describe(self):
        super().describe() # Call parent class describe method
        print(f"I am also a circle with radius {self.radius}.")

circle1 = Circle("red", 5)
circle1.describe()
circle1.calculate_area() # Access specific child class method
```

Encapsulation

- Encapsulation is a fundamental OOP concept that involves wrapping data and methods within a single unit.
- Controlling access to data and methods
- Protecting internal implementation details
- Using private attributes and methods (prefixed with `__`)



Encapsulation
in
python

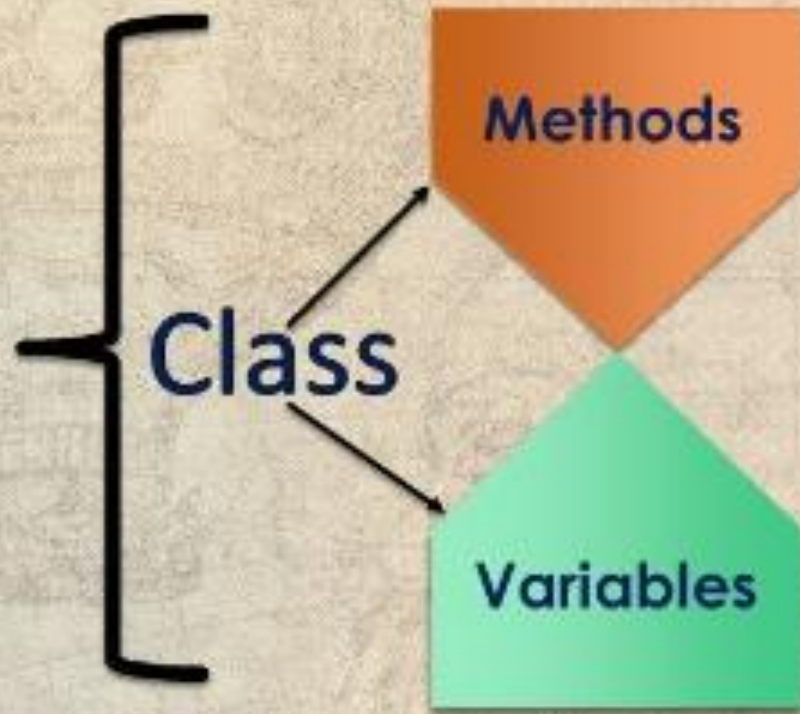
Why Encapsulation?

- **Pythonic Convention:** Emphasizes the use of encapsulation for clean and organized code.
- **Dynamic Typing:** Encapsulation helps maintain data integrity in Python's dynamically-typed environment.
- **Code Readability:** Promotes clear and readable code by encapsulating related functionalities.



Python Encapsulation

Encapsulation



- **Access Modifiers**

- **Public**
- **Private**
- **Protected**

Python

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance # Private attribute

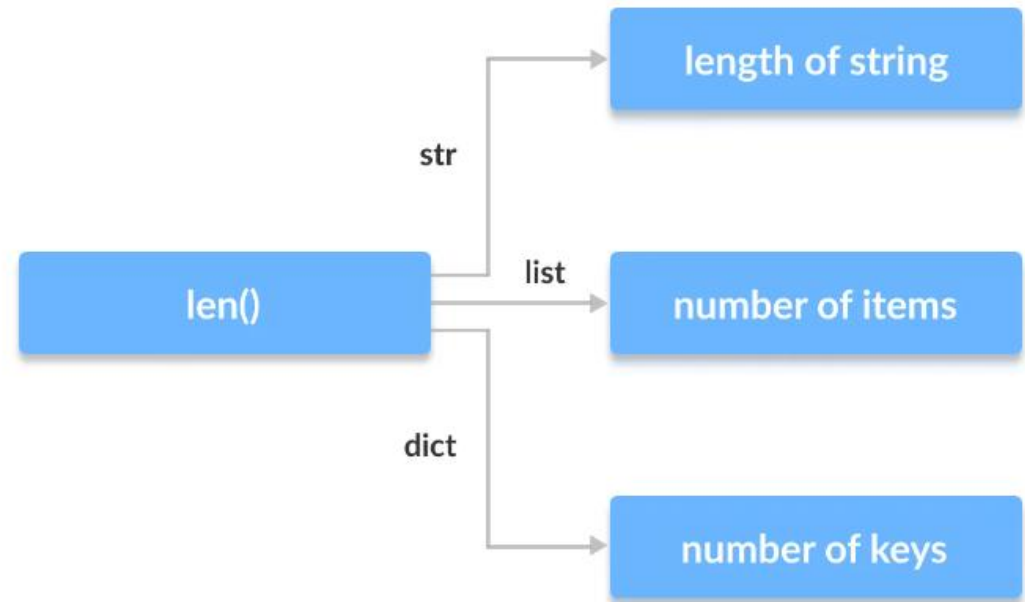
    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            print(f"Deposited {amount}, new balance: {self.__balance}")
        else:
            print("Invalid deposit amount.")

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
            print(f"Withdrawn {amount}, new balance: {self.__balance}")
        else:
            print("Insufficient funds.")

account = BankAccount(100)
account.deposit(50) # Public method used to access private data
account.withdraw(75) # Another public method with controlled access

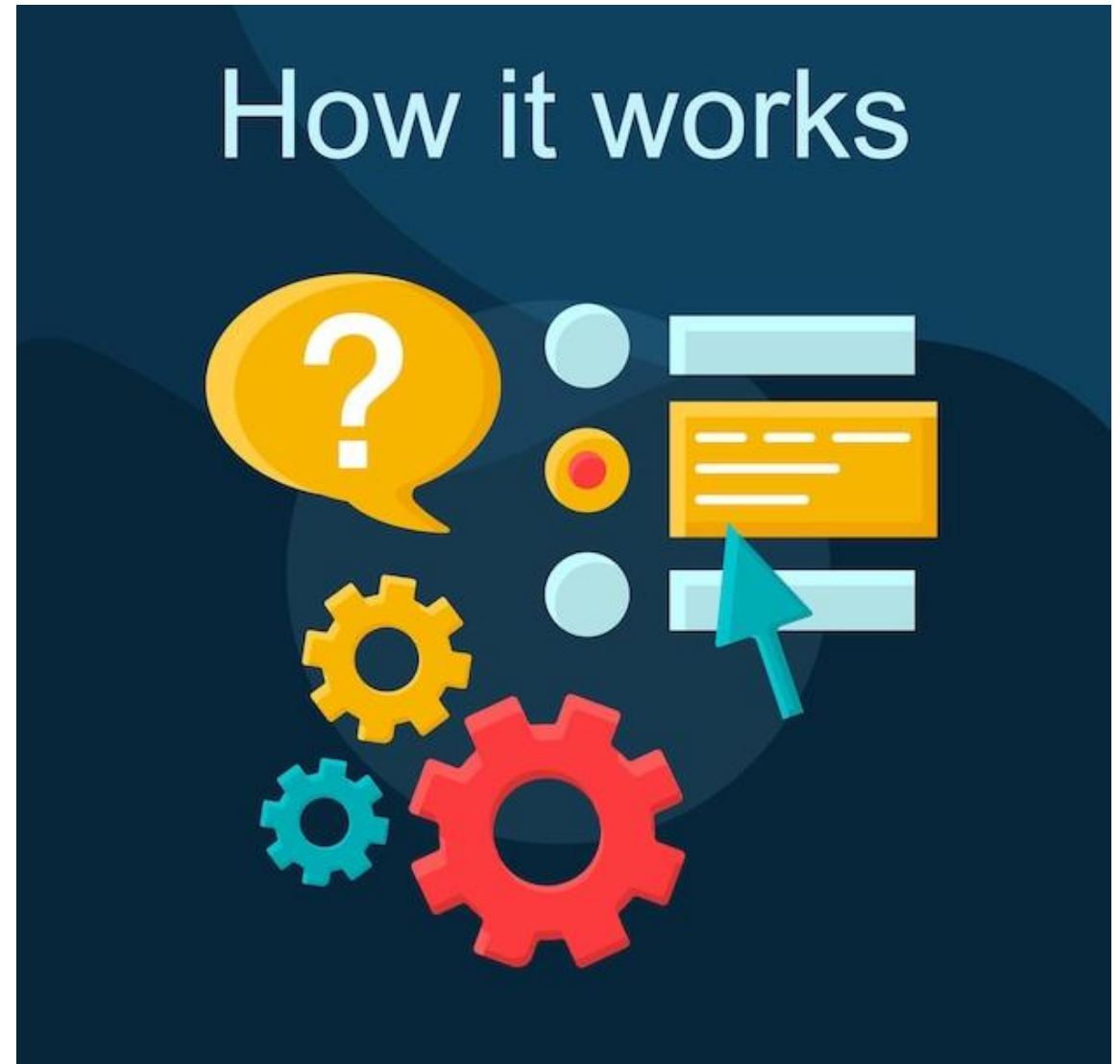
# Trying to directly access private attribute raises an error
# print(account.__balance) # AttributeError: 'BankAccount.__balance' is not accessible
```


Polymorphism



How Polymorphism Works

- **Method overriding:** Subclasses can redefine methods inherited from their parent class to provide their own behavior for the same operation.
- **Duck typing:** Focuses on whether an object has the necessary method or functionality to fulfill a task, regardless of its specific class.



Function Overloading

python

```
def add(a, b, c=0):  
    return a + b + c  
  
result1 = add(1, 2)  
result2 = add(1, 2, 3)  
  
print(result1) # Output: 3  
print(result2) # Output: 6
```

Operator Overloading

- Python allows custom behavior for operators through operator overloading. This is achieved by defining special methods in a class.
- In this example, the Point class defines the `__add__` method, allowing instances of the class to use the `+` operator for custom addition.

python

```
# Operator Overloading Example

class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)

# Usage
point1 = Point(1, 2)
point2 = Point(3, 4)
result = point1 + point2

print(result.x, result.y) # Output: 4 6
```

Method Overriding

python

```
# Method Overriding Example
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self):
        print("Dog barks")

# Usage
animal = Animal()
dog = Dog()

animal.speak() # Output: Animal speaks
dog.speak()    # Output: Dog barks
```

- Method overriding occurs when a subclass provides a specific implementation of a method already defined in its superclass.
- In this example, the Dog class overrides the speak method inherited from the Animal class. When calling speak on a Dog object, the overridden method in the Dog class is executed.

Benefits of Polymorphism

- **Flexibility:** Code can be written to work with different types of objects without needing specific knowledge of each one.
- **Reusability:** Common operations can be defined for a base class and implemented differently in its subclasses, reducing code duplication.
- **Dynamic binding:** Allows the program to determine the exact implementation of a method at runtime based on the object's type.



Difference between Procedural Programming and Object-Oriented Programming

FEATURE	PROCEDURAL PROGRAMMING (POP)	OBJECT-ORIENTED PROGRAMMING (OOP)
Structure	Linear, step-by-step execution	Object-based, modular structure
Focus	Functions and data flow	Objects, their interactions, and data
Key Concepts	Functions, data, global variables	Classes, objects, inheritance, polymorphism, encapsulation
Approach	Top-down	Bottom-up
Advantages	Simpler to learn, efficient for smaller programs	Better organization, reusability, maintainability, real-world modeling
Suitability	Mathematical calculations, data processing, simple scripts	Large, complex software systems, GUIs, simulations, modeling
Data Security	Less secure	More secure due to encapsulation



Day-22

Python Course

Agenda

- List, Tuple, and Dictionary Comprehensions
- Lambda, Reduce, Filter, and Map functions

Comprehension: Creating collections with ease

- Comprehensions offer a compact way to build lists, tuples, and dictionaries based on existing sequences.
- They eliminate the need for explicit loops and offer improved readability.
- Three types:
 - List comprehension
 - Tuple comprehension
 - Dictionary comprehension



List Comprehension

Python

```
numbers = [1, 2, 3, 4, 5]
squares = []
for number in numbers:
    squares.append(number * number)

print(squares)
```

Python

```
squares = [number * number for number in numbers]  
print(squares)
```

Using list comprehension

- The comprehension iterates over each element in ***numbers***, squares it, and adds it to the new list of squares.
- The syntax is clear and concise, making the code easier to understand and maintain.

Tuple and Dictionary Comprehensions

- Similar to list comprehensions, you can also create tuples and dictionaries with concise constructions:

- Dictionary comprehension: `my_dict = {x: x**2 for x in numbers}` (creates a dictionary with numbers as keys and their squares as values)

Functional Programming with Lambda, Reduce, Map, and Filter

- Functional programming emphasizes using functions as building blocks and avoiding mutable states.
- Python offers several built-in functions that embody this approach.



Lambda Functions: Anonymous Functions on the Fly

- Imagine needing a small function for a specific task within another function. Instead of defining a separate function, you can use a lambda function:
- Here, the ***lambda*** keyword defines an anonymous function that takes one argument ***x*** and squares it. This function is used directly within the list comprehension, making the code more concise.

Python

```
square = lambda x: x * x  
squares = [square(x) for x in numbers]
```

Reduce: Accumulating Values through Repeatedly Applying a Function

Python

```
from functools import reduce
sum_of_squares = reduce(lambda a, b: a + b**2, numbers)
print(sum_of_squares)
```

Map: Applying a Function to Each Element in a Sequence

Python

```
doubled_numbers = map(lambda x: 2 * x, numbers)
print(list(doubled_numbers))
```

- Mapping a function to each element in a sequence is a common task. **map** applies the function to every element of an iterable and returns a new iterable with the results:
- This code applies the function **lambda x: 2 * x** to each element in **numbers**, creating a new list **doubled_numbers** with double the values.

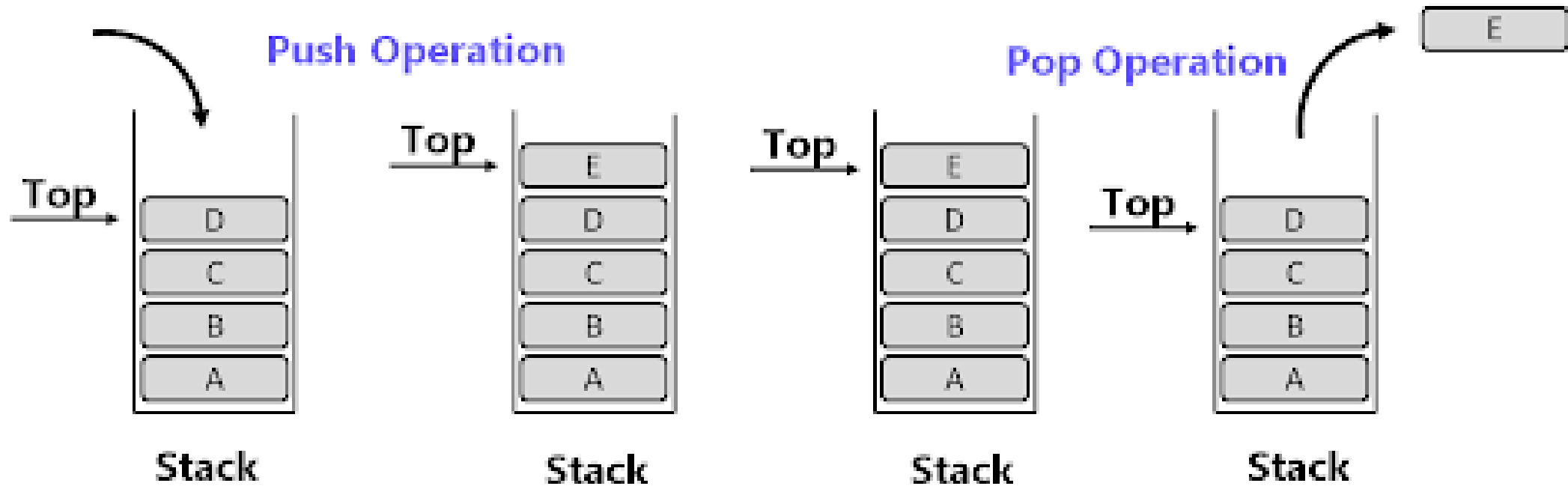
Filter: Selecting Elements Based on a Predicate

Python

```
even_numbers = filter(lambda x: x % 2 == 0, numbers)
print(list(even_numbers))
```

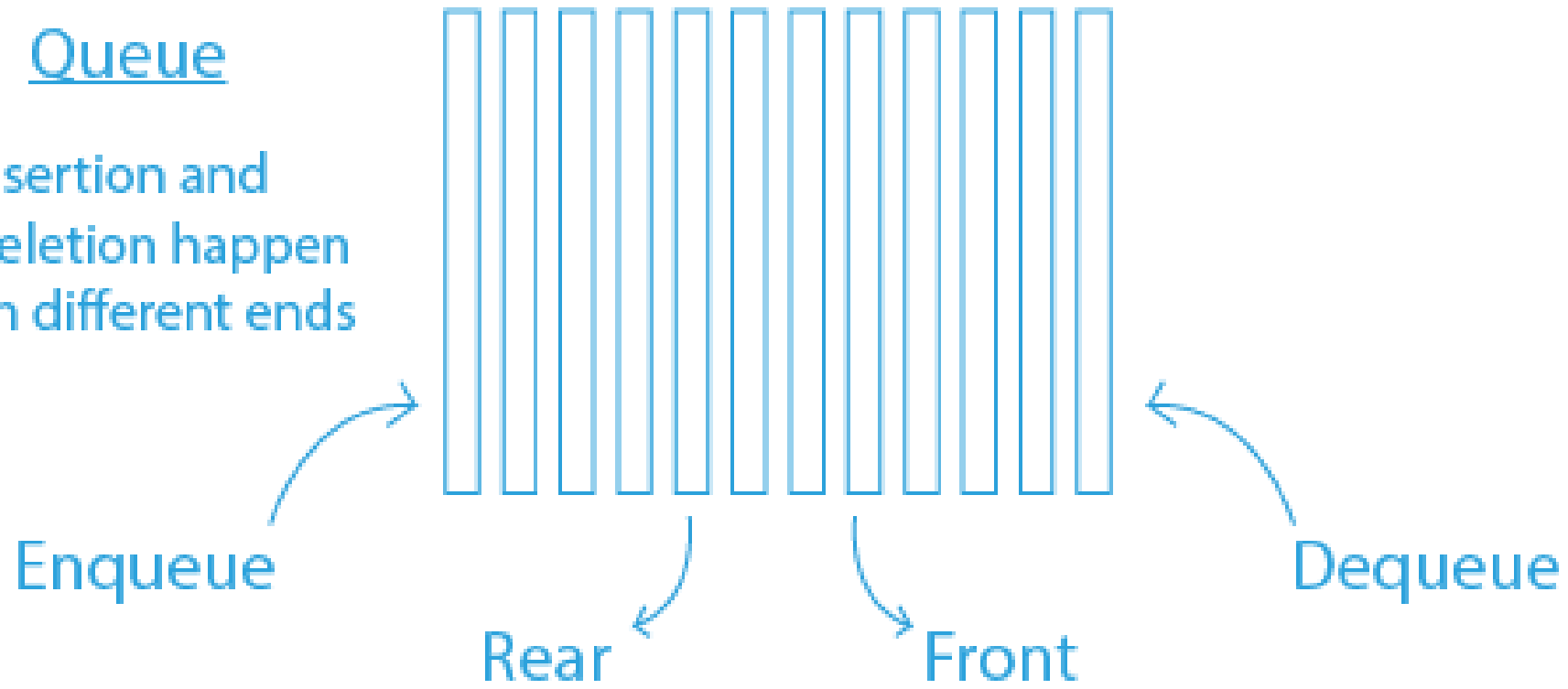
- Filtering elements based on a condition is another common operation. *filter* takes a function and an iterable, returning a new iterable containing only elements that satisfy the predicate:
- This code filters the *numbers* list using the predicate *lambda x: x % 2 == 0*, which checks if the number is even. The resulting *even_numbers* list contains only the even elements.

PUSH AND POP OPERATIONS IN STACK



Queue

Insertion and
Deletion happen
on different ends



First in First out

PYTHON PROJECTS FOR BEGINNERS



ADVANCED
CALCULATOR



UNIT
CONVERTER



AUDIO VIDEO
CONVERTER



YOUTUBE
DOWNLOADER



PASSWORD
GENERATOR



NUMBER GUESS
GAME



PRIME
FINDER



ROCK, PAPER,
SCISSORS GAME.



BINARY SEARCH
ALGORITHM.



WEBSITE
BLOCKER



COUNTDOWN
CLOCK AND TIMER



EMAIL
SLICER